National University
Of Computer and Emerging Sciences

Foundation of Advancement
Of Science and Technology

Lab Number:	11
Course Code:	CL2005
Course Title:	Database Systems - Lab
Semester:	Spring 2026
Agenda:	TCL & DCL
Instructor:	Muhammad Mehdi
Email:	muhammad.mehdi@nu.edu.pk
Office:	Rafaqat Lab

Table of Contents

1 Transaction Control Language (TCL)

Transaction Control Language (TCL) consists of SQL commands used to manage transactions in a database. These commands control how and when changes made by SQL statements are saved or undone.

A transaction is a sequence of one or more SQL operations executed as a single logical unit of work.

All transactions follow the **ACID properties**:

- **Atomicity:** All operations in a transaction are completed successfully, or none are applied.
- **Consistency:** The database remains in a valid state before and after the transaction.
- **Isolation:** Concurrent transactions do not interfere with each other.
- **Durability:** Once a transaction is committed, its changes are permanently stored.

In multi-user database systems, multiple users may access or modify the same data at the same time. Transactions ensure:

- Data integrity
- Prevention of partial updates
- Protection against conflicts and inconsistencies

However, some operations (such as **DDL commands** like `CREATE`, `DROP`, `ALTER`) often cause **implicit commits**, meaning they cannot be rolled back in many database systems.

Main TCL Commands:

- `START TRANSACTION`**:** Starts a new transaction
- `COMMIT`**:** Saves all changes permanently
- `ROLLBACK`**:** Undoes/reverses changes made since last commit or savepoint in a transaction
- `SAVEPOINT`**:** Sets a point within a transaction to roll back to

1.1 START TRANSACTION

It is a TCL command used to **begin a new transaction**. All SQL statements executed after this command are treated as part of a single transaction.

The changes made during the transaction are **temporary** and will not be permanently saved until a `COMMIT` is issued.

General Syntax:

```
START TRANSACTION
-- SQL statements
```

Example:

```
START TRANSACTION;

UPDATE employees
SET salary = salary + 500;

-- Changes are not yet permanent
```

1.2 COMMIT

It is a command used to permanently **save all changes** made during the current transaction.

Once a transaction is committed:

- Changes become **permanent**
- Changes become **visible to other users**
- They **cannot be undone**

General Syntax:

```
-- SQL statements
COMMIT;
```

Example:

```
START TRANSACTION;

UPDATE employees
SET salary = salary + 500;

COMMIT;

-- Changes are now permanently saved
```

1.3 ROLLBACK

It is a command used to **undo all changes** made in the current transaction and restore the database to its previous state.

It only works if the transaction has **not yet been committed**.

General Syntax:

```
-- SQL statements within a transaction
ROLLBACK;
```

Example:

```
START TRANSACTION;

UPDATE employees
SET salary = salary + 500;

ROLLBACK;

-- All changes are undone
```

1.4 SAVEPOINT

It is a command used to create a **checkpoint** (marker) within a transaction. It allows **partial rollback** instead of undoing the entire transaction.

General Syntax:

```
-- SQL statements within a transaction
SAVEPOINT savepoint_name;
-- SQL statements within a transaction
```

Example:

```
START TRANSACTION;

UPDATE employees
SET salary = salary + 500;
```

```
SAVEPOINT before_delete;

DELETE FROM employees;

-- No changes committed yet
```

1.5 ROLLBACK TO

It is a command used to **undo** only the changes made after a **specific savepoint**, while keeping earlier changes intact.

General Syntax:

```
-- SQL statements within a transaction
ROLLBACK TO savepoint_name;
-- SQL statements within a transaction
```

Example:

```
START TRANSACTION;

UPDATE employees
SET salary = salary + 500;

SAVEPOINT before_delete;

DELETE FROM employees;

ROLLBACK TO before_delete;

COMMIT;

-- Only the DELETE operation is undone
-- The UPDATE remains saved
```

2 Data Control Language (DCL)

Data Control Language (DCL) consists of SQL commands used to **manage user access and permissions** in a database system.

It ensures **data security** by controlling who can access the database and what operations they are allowed to perform.

DCL is mainly used by the Database Administrator (DBA) to enforce restricted and controlled access.

2.1 Users in MySQL

A user in MySQL is an account that can connect to and interact with the database server.

Each user is defined by:

- Username
- Host (the location from which the user is allowed to connect from)

Viewing Users:

```
-- Show all users
SELECT user, host, password FROM mysql.user;

-- Show current logged-in user
SELECT CURRENT_USER;
```

2.2 Managing Users

When working with users, two information is always required:

- **username:** Name assigned during user creation
- **host:** IP address or hostname (e.g., `localhost`, `%` for any host etc.)

2.2.1 Creating a User

Creates a new user account with authentication credentials.

General Syntax:

```
CREATE USER 'username'@'host' IDENTIFIED BY 'password';
```

Example:

```
CREATE USER 'mehdi'@'localhost' IDENTIFIED BY '1234';
```

2.2.2 Modifying a User

Used to change user properties such as password or authentication method.

General Syntax:

```
ALTER USER 'username'@'host' IDENTIFIED BY 'new_password';
```

Example:

```
ALTER USER 'mehdi'@'localhost' IDENTIFIED BY '5678';
```

2.2.3 Deleting a User

Removes a user account from the database server.

General Syntax:

```
DROP USER 'username'@'host';
```

Example:

```
DROP USER 'mehdi'@'localhost';
```

2.3 Privileges in MySQL

Privileges are permissions that define what actions a user can perform on database objects (databases, tables, etc.).

Main DCL Commands:

- `GRANT`**:** Assign privileges to users
- `REVOKE`**:** Remove privileges from users
- `FLUSH PRIVILEGES`**:** Reload privilege tables to apply changes immediately

2.3.1 Privilege Levels

Privileges can be applied at different levels:

Level	Description
Global	Applies to all databases and tables (`*.*`)
Database	Applies to all tables in a specific database
Table	Applies to a specific table or view

2.3.2 Granting Privileges

The `GRANT` command assigns permissions to a user.

General Syntax:

```
GRANT privilege_list
ON database.table
TO 'username'@'host';
```

Examples:

- Global privileges:

```
-- All permissions on all tables in all databases in the server
GRANT ALL
ON *.*
TO 'admin'@'localhost';
```

- Database-level privileges:

```
-- Complete access on all tables in 'school' database
GRANT ALL
ON school.*
TO 'student'@'localhost';
```

- Table-level privileges:

```
-- Full access on 'employees' table in 'school' database
GRANT ALL
ON school.employees
TO 'manager'@'localhost';
```

2.3.3 Removing Privileges

The `REVOKE` command removes previously granted permissions.

General Syntax:

```
REVOKE privilege_list
ON database.table
FROM 'username'@'host';
```

Examples:

```
REVOKE ALL
ON *.*
FROM 'admin'@'localhost';

REVOKE ALL
ON school.*
FROM 'student'@'localhost';

REVOKE ALL
ON school.employees
FROM 'manager'@'localhost';
```

2.3.4 Applying Privileges

The `FLUSH PRIVILEGES` command reloads the grant tables to ensure that changes take effect immediately.

Example:

```
-- Grant/Revoke commands

FLUSH PRIVILEGES;
```

2.3.5 Privileges Types

These are common privileges that can be granted to users:

Type	Description

ALL	All privileges
ALTER	Modify table structure
ALTER ROUTINE	Modify stored procedures/functions
CREATE	Create databases/tables
CREATE ROUTINE	Create stored procedures/functions
CREATE USER	Create new users
CREATE VIEW	Create views
DELETE	Delete rows
DROP	Remove databases/tables
EXECUTE	Execute stored procedures/functions
GRANT OPTION	Allow granting privileges to others
INSERT	Insert data
SELECT	Read data
SHOW DATABASES	View database list
TRIGGER	Create triggers
UPDATE	Modify existing rows

Examples:

```
-- Grant SELECT, INSERT on a database
GRANT SELECT, INSERT ON school.* TO 'student'@'localhost';

-- Grant ALL privileges on a specific table
GRANT ALL ON school.employees TO 'manager'@'localhost';

-- Revoke privileges
REVOKE INSERT ON school.* FROM 'student'@'localhost';
```

```
-- Apply changes
FLUSH PRIVILEGES;
```

2.3.6 Viewing User Privileges

Following statements are commonly used to check what permissions a user currently has.

Examples:

- Show privileges using SQL command:

```
SHOW GRANTS FOR 'student'@'localhost';
```

- Query privilege tables:

```
SELECT user, host, Select_priv, Insert_priv, Update_priv
FROM mysql.user;
```